

Con el PDF, la fecha límite de **30 días** y vuestro nivel básico, yo plantearía la práctica como un **MVP sólido**, no como un juego complejo. El enunciado valora más un sistema claro, correcto y bien diseñado que uno complejo pero desorganizado, y además exige JavaFX, JSON, UML, pruebas, diario de IA y estructuras propias.

1. Decisión estratégica: qué hacer y qué NO hacer

Haced obligatoriamente

El juego mínimo debería tener:

1. **Grafo de habitaciones propio.**
2. **Cada habitación como matriz propia de celdas.**
3. **Jugador con vida, ataque, defensa, movimiento e inventario.**
4. **Objetos simples:** poción, llave, arma.
5. **Enemigos simples:** se acercan al jugador y atacan si están al lado.
6. **Puertas entre habitaciones** y una puerta final de salida.
7. **BFS para calcular casillas alcanzables** dentro de la habitación.
8. **BFS en el grafo** para informar del número mínimo de habitaciones hasta la salida.
9. **Guardar/cargar JSON.**
10. **Interfaz JavaFX sencilla con GridPane.**
11. **Logs, excepciones y pruebas JUnit.**
12. **Memoria, UML, bocetos, diario de IA y vídeo.**

El PDF exige requisitos funcionales como grafo, matriz de habitaciones, inventario, combate, victoria/derrota, JSON de configuración y JSON de estado. También exige interfaz JavaFX con matriz, panel del jugador, acciones disponibles, registro de eventos y elementos interactivos básicos.

No hagáis, salvo que os sobre tiempo

No implementaría al principio:

- Turnos propios por habitación.
- Dijkstra con costes variables.
- Árbol de habilidades complejo.
- Muchas armas/equipamiento por partes del cuerpo.
- IA avanzada de enemigos.
- Gráficos elaborados.
- Enemigos que cambian de habitación.
- Animaciones JavaFX.

- Sistema complejo de estados alterados.

Con nivel básico, esas ampliaciones os pueden romper la planificación.

2. Arquitectura recomendada

Separad el proyecto en paquetes así:

src/main/java

```
└─ app
|   └─ MainApp.java
└─ model
    ├── GameState.java
    ├── Player.java
    ├── Enemy.java
    ├── Room.java
    ├── Cell.java
    ├── Position.java
    ├── Item.java
    ├── Weapon.java
    ├── Potion.java
    ├── Key.java
    └─ Door.java
└─ structures
    ├── MyList.java
    ├── MyLinkedList.java
    ├── MyQueue.java
    ├── MyStack.java
    ├── MyCircularList.java
    ├── MyTree.java
    ├── MyGraph.java
    └─ MyMatrix.java
└─ service
```

```

|   ├── GameEngine.java
|   ├── MovementService.java
|   ├── CombatService.java
|   ├── InventoryService.java
|   ├── PathFindingService.java
|   └── TurnService.java
├── persistence
|   ├── GameLoader.java
|   ├── GameSaver.java
|   └── JsonMapper.java
├── ui
|   ├── GameController.java
|   ├── RoomView.java
|   └── PlayerPanelView.java
├── exception
|   ├── InvalidMoveException.java
|   ├── InvalidActionException.java
|   └── PersistenceException.java
└── log
    └── GameLog.java

```

La clave es que **JavaFX no debe contener la lógica del juego**. JavaFX solo pinta lo que diga GameEngine. Esto os facilitará las pruebas y encaja con la evaluación, que valora separación entre lógica e interfaz.

3. Estructuras propias que deberíais implementar

El PDF prohíbe ArrayList, HashMap, LinkedList y estructuras estándar equivalentes, y exige estructuras propias como listas, pilas, colas, listas circulares, árboles y grafos. Además, avisa que usar estructuras de Java o librerías externas para las estructuras evaluadas puede suponer un 0 en Estructuras de Datos.

Estructura propia **Uso en el juego**

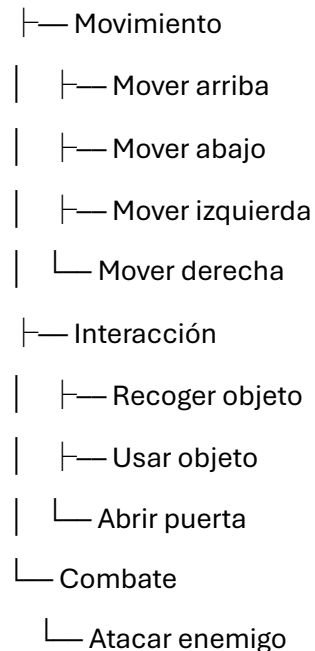
MyLinkedList<T> Inventario, lista de enemigos, lista de objetos, lista de habitaciones

Estructura propia Uso en el juego

MyQueue<T>	Turnos y BFS
MyStack<T>	Opcional: historial de acciones o ruta calculada
MyCircularList<T>	Orden de turnos jugador/enemigos
MyGraph<Room>	Mapa de habitaciones conectadas
MyMatrix<Cell>	Tablero interno de cada habitación
MyTree<Action>	Árbol sencillo de acciones posibles del turno

Para cumplir el requisito de árbol sin complicaros, haría un árbol de acciones:

Acción



No tiene que ser sofisticado. Lo importante es poder justificarlo en la memoria.

4. Reparto entre tres personas

Os propongo un reparto por **responsabilidad técnica**, pero con integración semanal. Así podéis trabajar en paralelo sin pisaros.

Persona A — Estructuras de datos y algoritmos

Responsable de:

- MyLinkedList
- MyQueue
- MyStack

- MyCircularList
- MyTree
- MyGraph
- MyMatrix
- BFS de casillas alcanzables.
- BFS de habitaciones hasta salida.
- Tests JUnit de estructuras.
- Justificación de costes en la memoria.

Esta persona lleva la parte más crítica para Estructuras de Datos.

Persona B — Modelo y lógica del juego

Responsable de:

- Player
- Enemy
- Room
- Cell
- Item, Weapon, Potion, Key
- Door
- GameState
- GameEngine
- Movimiento.
- Inventario.
- Combate.
- Condiciones de victoria y derrota.
- Logs de eventos.
- Excepciones de lógica.

Esta persona conecta las estructuras de A con las reglas reales del juego.

Persona C — JSON, JavaFX, documentación y coordinación visual

Responsable de:

- Cargar configuración inicial desde JSON.
- Guardar estado de partida en JSON.
- Cargar partida guardada.

- Interfaz JavaFX.
- GridPane de habitación.
- Panel de jugador.
- Botones de acciones.
- Registro visual de eventos.
- Bocetos de interfaz.
- Capturas para memoria.
- Preparación inicial del vídeo.

Aunque esta persona tenga documentación, también tiene bastante programación: persistencia e interfaz.

5. Plan de 30 días

Días 1-3 — Diseño y decisiones obligatorias

Objetivo: salir con una visión común antes de programar.

Tareas:

- Crear repositorio GitHub.
- Decidir reglas exactas del juego.
- Hacer boceto de pantalla.
- Definir clases principales.
- Definir casos de uso.
- Definir JSON de configuración.
- Definir JSON de guardado.
- Crear documento de decisiones.

Decisiones recomendadas:

Tema	Decisión recomendada
Habitaciones	4 habitaciones
Tamaño	5x5 o 6x6
Jugador	Vida, ataque, defensa, movimiento
Enemigos	1 o 2 por habitación
Objetos	Poción, llave, arma

Tema	Decisión recomendada
Puertas	Algunas libres, una con llave
Objetivo	Llegar a la habitación final y abrir salida
Movimiento enemigo	Se acerca al jugador por BFS simple
Combate	Fórmula del enunciado
Turnos	Turnos globales, sin turnos por habitación

Entregables de esta fase:

- Boceto de interfaz.
- Lista de requisitos.
- Primer diagrama de clases.
- Primer JSON de ejemplo.
- Contrato básico de estructuras.

El PDF indica expresamente que hay que diseñar pantallas antes de programar y recomienda hacerlo como primer paso para alinear al equipo.

Días 4-9 — Estructuras propias y modelo base

Objetivo: tener el núcleo de datos funcionando sin JavaFX.

Persona A:

- Implementar `MyLinkedList<T>`.
- Implementar `MyQueue<T>`.
- Implementar `MyMatrix<T>`.
- Empezar `MyGraph<T>`.
- Tests JUnit.

Persona B:

- Implementar `Position`, `Cell`, `Room`.
- Implementar `Player`, `Enemy`.
- Implementar `Item`, `Potion`, `Weapon`, `Key`.
- Implementar reglas básicas de vida, ataque y defensa.

Persona C:

- Preparar proyecto JavaFX.
- Crear estructura de carpetas.

- Preparar JSON de configuración inicial.
- Investigar librería JSON permitida por el profesor, por ejemplo Gson o Jackson.
- Crear primera vista vacía.

Resultado al día 9:

- Se puede crear una habitación en código.
- Se puede crear un jugador.
- Se puede crear una matriz de celdas.
- Hay listas propias funcionando.
- Hay tests básicos.

Días 10-15 — Movimiento, grafo y turnos

Objetivo: que el juego funcione por consola o mediante tests, todavía sin depender de JavaFX.

Persona A:

- Terminar `MyGraph<Room>`.
- Implementar BFS en grafo de habitaciones.
- Implementar BFS para casillas alcanzables dentro de la habitación.
- Implementar `MyCircularList<T>` para orden de turnos.
- Tests de BFS.

Persona B:

- Implementar `MovementService`.
- Implementar cambio de habitación.
- Implementar `TurnService`.
- Implementar condiciones de victoria/derrota.
- Implementar log de eventos.

Persona C:

- Implementar carga de configuración JSON.
- Convertir JSON a objetos del modelo.
- Preparar vista JavaFX de matriz con `GridPane`.
- Mostrar jugador y celdas básicas.

Resultado al día 15:

- El jugador puede moverse dentro de una habitación.
 - El jugador puede cambiar de habitación.
 - El grafo calcula distancia mínima hasta salida.
 - Hay una configuración JSON cargable.
 - La interfaz ya puede pintar una habitación.
-

Días 16-21 — Inventario, combate, enemigos y guardado

Objetivo: cerrar la jugabilidad mínima.

Persona A:

- Implementar MyTree<Action>.
- Implementar tests de árbol.
- Revisar costes de operaciones para documentarlos.
- Ayudar a integrar estructuras donde falten.

Persona B:

- Implementar recoger objeto.
- Implementar usar poción.
- Implementar equipar arma o llave de forma simple.
- Implementar atacar enemigo.
- Implementar ataque enemigo.
- Implementar movimiento enemigo hacia jugador.

Persona C:

- Implementar guardar partida en JSON.
- Implementar cargar partida guardada.
- Añadir botones JavaFX: mover, recoger, usar, atacar, guardar, cargar.
- Mostrar inventario constantemente.
- Mostrar log de eventos.

Resultado al día 21:

- Ya tenéis un juego jugable de principio a fin.
- Se puede ganar.
- Se puede perder.
- Se puede guardar.

- Se puede cargar.
- Se ve en JavaFX.

Días 22-25 — Pruebas, excepciones y estabilización

Objetivo: dejar de añadir cosas nuevas y corregir errores.

Tareas comunes:

- Pruebas JUnit de estructuras.
- Pruebas JUnit de movimiento.
- Pruebas JUnit de combate.
- Pruebas JUnit de inventario.
- Pruebas JUnit de carga/guardado JSON.
- Probar movimientos inválidos.
- Probar ataques inválidos.
- Probar JSON corrupto.
- Probar victoria.
- Probar derrota por vida.
- Probar derrota por turnos.

El PDF exige pruebas de estructuras, lógica del juego, carga/guardado JSON, logs y gestión de excepciones.

Días 26-28 — Memoria, UML, diario de IA y vídeo

Objetivo: preparar la entrega, no programar más.

Persona A:

- Sección de estructuras de datos.
- Justificación de costes.
- Capturas de tests.
- Explicación de BFS, cola, grafo y matriz.

Persona B:

- Sección de modelo de dominio.
- Reglas del juego.
- Casos de uso.

- Contratos e invariantes.
- Crítica del proyecto.

Persona C:

- Sección de interfaz.
- JSON.
- Bocetos.
- Diario de IA.
- Montaje del vídeo.

UML obligatorio:

- Diagrama de casos de uso.
- Diagrama de clases.
- Diagrama de secuencia.
- Diagrama de estados.
- Diagrama de actividad.

El PDF exige esos diagramas UML, además de documento de diseño, JSON de ejemplo, pruebas, diario de IA y bocetos.

Días 29-30 — Congelación y entrega

Objetivo: no tocar funcionalidad salvo errores graves.

Checklist final:

- El proyecto compila desde cero.
- El ZIP contiene todo.
- El repositorio GitHub está actualizado.
- Hay una entrega para cada asignatura.
- La memoria tiene portada con los tres nombres.
- La memoria incluye “crítica del proyecto”.
- El vídeo muestra todas las funcionalidades.
- El vídeo incluye a los tres miembros.
- Los JSON de ejemplo están incluidos.
- Los tests se ejecutan.
- No hay ArrayList, HashMap, LinkedList ni equivalentes en partes evaluadas.

- Se ve el inventario constantemente.
- Se muestra el log al final de la partida.
- Hay gestión de excepciones.

El PDF exige memoria PDF con sección “crítica del proyecto”, vídeo explicativo con todos los alumnos y portada con los nombres de quienes han participado.

6. Orden real de implementación

Este sería el orden que yo seguiría:

1. Crear repositorio.
2. Crear paquetes.
3. Crear estructuras propias.
4. Crear modelo básico.
5. Crear matriz de habitación.
6. Crear grafo de habitaciones.
7. Crear jugador.
8. Crear movimiento dentro de habitación.
9. Crear cambio de habitación.
10. Crear cálculo de casillas alcanzables.
11. Crear cálculo de camino mínimo a salida.
12. Crear objetos.
13. Crear inventario.
14. Crear enemigos.
15. Crear combate.
16. Crear turnos.
17. Crear condiciones de victoria/derrota.
18. Crear logs.
19. Crear carga JSON.
20. Crear guardado JSON.
21. Crear interfaz JavaFX.
22. Conectar interfaz con GameEngine.
23. Añadir pruebas.

- 24. Hacer UML.
 - 25. Hacer memoria.
 - 26. Grabar vídeo.
-

7. Mínimo tablero recomendado

Para no complicaros:

Habitación 1: Inicio

- Jugador empieza aquí
- Hay una poción
- Hay una puerta a Habitación 2

Habitación 2: Combate

- Hay un enemigo
- Hay una llave
- Puerta a Habitación 3

Habitación 3: Decisión

- Hay un enemigo
- Hay una puerta a Habitación 4
- Hay una puerta cerrada que requiere llave

Habitación 4: Salida

- Hay puerta final al exterior

Grafo:

H1 -- H2 -- H3 -- H4

Así cumplís grafo, habitaciones, movimiento, objetos, llave, enemigo, combate y victoria sin crear un mapa enorme.

8. Qué debería hacer cada uno cada semana

	Semana	Persona A	Persona B	Persona C
1		Estructuras base	Modelo base	Bocetos, JSON inicial, proyecto JavaFX
2		Grafo, matriz, BFS	Movimiento, turnos	Carga JSON, primera pantalla
3		Árbol, tests estructuras	Inventario, combate, enemigos	Guardar/cargar, UI completa
4		Documentar estructuras	Pruebas lógica, memoria	UML, diario IA, vídeo, entrega

9. Reglas de Git para que no os estorbéis

Usad ramas simples:

main

develop

feature/estructuras

feature/logica-juego

feature/javafx-json

feature/tests

feature/documentacion

Normas:

- Nadie programa directamente en main.
 - Integración cada 2 o 3 días.
 - Cada pull request debe compilar.
 - Cada clase importante debe tener al menos una prueba.
 - Si alguien cambia una interfaz común, avisa al grupo antes.
-

10. Entregable mínimo viable

El objetivo realista para vosotros debería ser:

Un juego sencillo, con 4 habitaciones, movimiento por matriz, grafo propio, inventario básico, combate básico, enemigos simples, JSON, JavaFX clara, logs, pruebas y documentación completa.

Eso tiene muchas más probabilidades de sacar buena nota que intentar hacer un RPG grande y acabar con código desordenado.

